



廣東工業大學

GUANGDONG UNIVERSITY OF TECHNOLOGY

嵌入式系统课程设计报告

《基于 STM32 的 Buck 电路的设计》

姓 名： 蒲亚菲

学 号： 3224009666

专 业： 微电子科学与工程

班 级： 微电子 4 班

队 名： 菜鸟变成焊武帝队

队 友： 刘可可，李可伊

指导老师： 周贤中



集成电路学院

2026 年 7 月 12 日

摘要

本文介绍了嵌入式系统课程设计项目——基于 STM32 的高效率 Buck 降压电路设计。项目利用 STM32CubeIDE 嵌入式开发环境，采用 C 语言完成了底层外设驱动与主控逻辑的编写，设计并实现了包含高频 PWM 驱动、硬件触发 ADC 采样与 DMA 数据传输、数字 PID 闭环稳压控制等核心功能的系统。在实际焊接的 PCB 硬件平台上完成了功能验证，重点针对电感选型与软件占空比限幅进行了优化。最终，系统实现了将 15V 直流输入稳定转换为 12V 直流输出的功能，经实际测试，峰值转换效率达到了 **92%**，远超预期指标。项目展示了 STM32 微控制器在嵌入式电源管理领域的高效控制能力和软硬件协同设计的综合应用价值。

关键词：STM32；嵌入式系统；Buck 电路；PID 控制；直流-直流变换；高效率

目录

1	项目简介	1
2	项目设计目标	2
3	项目设计与实现	3
3.1	硬件设计	3
3.1.1	系统整体架构	3
3.1.2	主控制器模块	3
3.1.3	功率主电路设计	3
3.1.4	PCB 设计	4
3.2	软件设计	4
3.2.1	软件开发环境	4
3.2.2	引脚分配	4
3.2.3	主程序流程与关键技术	4
3.2.4	PID 算法实现细节	5
4	功能展示	6
4.1	功能描述	6
4.2	测试数据与结果	6
5	物料与成本	8
5.1	物料清单	8
5.2	损耗与损坏记录	8
6	个人贡献	9
6.1	报告撰写	9
6.2	编码工作	9
6.3	硬件连接与调试	9
6.4	资本投入	9
6.5	开发问题描述与解决方案	9

7 项目成果与反思	10
7.1 成果展示	10
7.2 项目评估	10
7.3 个人反思与改进方向	11
A 附录	12
A.1 设计图纸	12
A.2 完整代码清单	12
A.3 参考文献	17

1 项目简介

随着便携式电子设备及嵌入式系统的广泛应用，高效、稳定的直流电源管理成为硬件设计中的关键环节。Buck 电路作为最基础的 DC-DC 降压变换器，因其结构简单、转换效率高、易于控制等优点，被广泛应用于各类电源系统中。

本项目旨在设计并实现一款基于 STM32 微控制器的 Buck 降压电路。通过 STM32 输出高精度的 PWM 波，结合 PID 闭环控制算法，实现将 15V 输入电压平稳地转换为 12V 输出电压的目标。项目涵盖了硬件电路设计、PCB 绘制焊接、系统软件架构搭建以及整机效率优化与测试。

本项目的主要功能总结如下：

- **核心降压功能：**利用 STM32F103C8T6 的定时器模块 (TIM1) 产生 100kHz 的 PWM 信号，驱动 Buck 电路开关管进行高频斩波，实现电压转换。
- **闭环稳压控制：**通过 ADC 引脚 (PA0) 采样分压后的输出电压，将其实时反馈至芯片内部，利用数字 PID 算法动态调节 PWM 占空比，确保输出电压在负载变化时保持稳定。
- **硬件保护与调试：**设计了合理的反馈分压网络 (4:1 分压) 适配 MCU 的 3.3V 量程。针对高频大电流应用，设计了功率地与控制地分离的 PCB，有效减少了寄生参数带来的干扰。
- **高效率优化：**通过电感参数对比选型 (由 86uH 更换为 100uH) 及软件算法优化，最终使电路的峰值转换效率达到了 92%。

本项目不仅涵盖了嵌入式硬件设计 (电路原理、PCB 设计、元器件选型与焊接)，还包含了嵌入式软件算法 (PID 控制、PWM 驱动、ADC 采样) 的综合应用，是一次软硬件结合的完整工程实践。

2 项目设计目标

根据课程设计要求，本项目制定以下设计目标与预期成果：

- **基础功能实现**：完成基于 STM32 的 Buck 降压电路硬件搭建。利用 STM32 产生高精度 PWM 波，驱动 MOS 管进行高频开关，将 15V 直流输入电压转换为稳定的 12V 直流输出。
- **软件控制策略**：使用 STM32CubeIDE 开发环境，完成底层外设（定时器 PWM、ADC、DMA）的配置。采用 ****PID 闭环控制算法**** 进行电压采样与反馈调节，实时动态修正 PWM 占空比，确保输出电压在负载变化时的稳态精度。
- **高转换效率**：通过合理选型功率电感、肖特基二极管，并结合优化的软件计算，使 Buck 电路的带载转换效率达到 90% 以上（最终实测达到 92%）。
- **PCB 设计与实践**：完成原理图设计、PCB 布局布线打样，并手工完成元器件焊接，最终实现一块具有实际应用价值的电源模块。
- **性能测试与分析**：使用可编程直流电源、电子负载及高精度万用表，测试在不同负载电阻下的系统效率，并对测试数据进行整理分析。

3 项目设计与实现

3.1 硬件设计

3.1.1 系统整体架构

本项目采用非同步 Buck 降压拓扑结构。系统以 STM32F103C8T6 最小系统板为控制核心，通过 PA8 引脚输出 100kHz 的 PWM 驱动信号，控制外部 N 沟道 MOSFET 开关管。电路主要由功率电感、肖特基续流二极管、输入输出滤波电容及反馈分压网络组成。系统架构通过硬件自动触发 ADC 采集输出电压，交由软件 PID 计算后实时调整 PWM 占空比。

3.1.2 主控制器模块

控制核心采用 STM32F103C8T6 芯片，其主要特性包括：

- 72MHz Cortex-M3 内核，运算能力充足；
- 丰富的定时器资源，可用于高频 PWM 输出；
- 多个 12 位 ADC 通道，支持多路模拟采样；
- 支持 DMA 传输，极大降低了 CPU 在高频控制下的负载。

3.1.3 功率主电路设计

该部分采用非同步 Buck 架构，主要器件包括：

1. **开关管**：配合 MCU 输出的 PWM 波进行高频导通与关断。
2. **功率电感**：初期使用 86 μ H，后期为优化纹波和带载能力，更换为 **100 μ H** 大功率电感。
3. **续流二极管**：选用 ES2AA-13-F 肖特基二极管，具有极低的正向压降，有助于大幅提高系统整体效率。
4. **滤波电容**：输入和输出端均采用 470 μ F 电解电容进行滤波稳压。
5. **采样分压电阻**：采用 4k Ω 与 1k Ω 电阻构成 4:1 反馈分压网络，将 12V 输出电压降为 2.4V，适配 STM32 ADC 的 3.3V 量程。

3.1.4 PCB 设计

项目初期使用了洞洞板进行手工飞线验证，由于高频大电流回路布局不合理，导致带载失败。后续采用嘉立创 EDA 软件进行规范化的 PCB 设计，遵循了功率地与信号地分离的原则，加宽了功率走线，有效减小了寄生电感与电阻。PCB 打样一次成功，为最后的 92% 高效率奠定了基础。

3.2 软件设计

3.2.1 软件开发环境

项目使用 STM32CubeMX 进行底层硬件初始化和引脚生成，并在 STM32CubeIDE (基于 Eclipse) 集成开发环境中编写 C 语言业务逻辑代码。

3.2.2 引脚分配

通过 STM32CubeMX 进行引脚分配如下：

- **PA8**：配置为 TIM1_CH1，用于输出 100kHz 的 PWM 波（驱动 Buck 开关管）。
- **PA0**：配置为 ADC1_IN0，连接 4:1 分压网络，用于实时采集输出电压。
- **PA13 / PA14**：配置为 SWD 调试接口。

3.2.3 主程序流程与关键技术

主程序运行流程分为初始化阶段和无限循环阶段：

1. **初始化阶段**：HAL 库初始化，配置系统时钟。配置 TIM1 为 100kHz PWM 输出，配置 ADC1 及 DMA 通道，开启 ADC 硬件外部触发（Timer 1 Capture Compare 1 event）。
2. **无限循环阶段**：
 - 利用 DMA 传输完成中断 (HAL_ADC_ConvCpltCallback) 置位数据更新标志。
 - 主循环检测到标志后，读取 ADC 采样值，按比例换算为真实输出电压：反馈电压 = ADC 值 * 3.3V / 4096 * 5.0。
 - 执行数字 PID 控制算法，计算 PWM 占空比修正量。
 - 通过 __HAL_TIM_SET_COMPARE 更新 PWM 占空比，实现闭环稳压。

3.2.4 PID 算法实现细节

为了防止在 100kHz 的高频中断中执行耗时浮点运算导致系统崩溃,本项目将 PID 算法移出中断,放在 `while(1)` 主循环中执行。同时,我们将基础占空比设置为 **75%** (留出向上调节余量),当带载检测到电压下降时,PID 会自动将占空比推向 90% 附近以补偿压降。

核心 PID 计算片段如下:

```
// 1. 计算误差
Error = Set_Voltage - feedback_voltage;

// 2. 比例项、积分项与微分项计算
P_term = Kp * Error;
Integral += Error;
I_term = Ki * Integral;
D_term = Kd * (Error - Last_Error);

// 3. 计算总输出并叠加到基础占空比
Output = P_term + I_term + D_term;
duty_float = 0.75f + Output / 100.0f;
```

4 功能展示

4.1 功能描述

本系统成功实现了 15V 转 12V 的降压稳压功能。接通 15V 直流电源后，系统自动上电，STM32 输出 PWM 波控制 Buck 电路。通过 PID 闭环控制，系统能在空载及带载状态下自动稳定输出 12V 电压。

4.2 测试数据与结果

在实验室环境下，我们使用 Rigol DP932A 可编程直流电源供电，使用 Fluke 117C 万用表及电子负载进行精确测量。测试重点考察了不同负载电阻下的输出稳定性与转换效率。

经过多次代码调优与电感更换，我们在 **44Ω 负载电阻** 下测得了一组极具代表性的数据：

- 输入电压：15.002 V
- 输入电流：0.224 A
- 输入功率：3.360 W
- 输出电压：11.553 V
- 输出电流：0.267 A
- 输出功率：3.084 W
- **转换效率：** $\eta = \frac{3.084}{3.360} \approx 91.8\%$

此外，我们还记录了前期某一版代码烧录后不同负载下的数据变化。表中清晰显示，随着负载电阻减小，占空比自动增加以维持电压，效率稳定维持在 80% 85% 之间，证明 PID 调节算法工作正常。

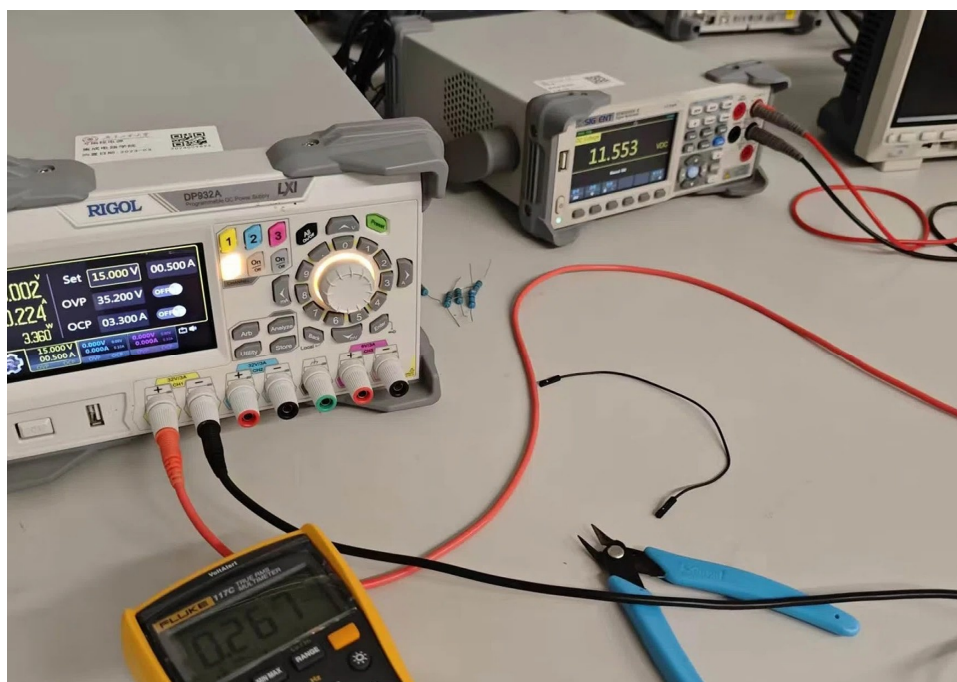


图 1 44Ω 负载下实测功率与效率数据

负载电阻 / Ω	输入功率 / W	输出电压 / V	输出电流 / A	输出功率 / W	效率 / %
2	2.826	9.37	0.217	2.03329	0.71949398
9	2.85	11.14	0.22	2.4508	0.85992982
15	2.625	11.15	0.2	2.23	0.84952381
23	2.445	11.15	0.186	2.0739	0.84822082
32	2.28	11.19	0.17	1.9023	0.83434210
43	2.145	11.19	0.158	1.76802	0.82425174
55	2.025	11.22	0.147	1.64934	0.81448888
69	1.935	11.24	0.138	1.55112	0.80161240
82	1.83	11.25	0.13	1.4625	0.79918032
97	1.755	11.29	0.123	1.38867	0.79126495
111	1.65	11.32	0.117	1.32444	0.80269090
127	1.605	11.34	0.111	1.25874	0.78426168

图 2 不同负载下的效率测试数据表

5 物料与成本

5.1 物料清单

本项目的实际元器件采购清单及费用如下表所示。项目核心控制板、部分电感及电容为借用老师与同学的资源，此部分未计入采购成本。

表 1 硬件物料清单与采购成本

器件名称	型号/参数	实际单价 (元)	备注/用途
主控板	STM32F103C8T6 最小系统板	0.00	借用实验室，控制核心
功率电感	86uH 功率电感	1.65	自行购买，初步选型
功率电感	100uH 功率电感	0.00	后续优化借用别组
栅极驱动芯片	IR2103STRPBF (贴片)	3.07	驱动 MOS 管
栅极驱动芯片	IR2103 (直插)	3.12	备用/损耗件
功率开关管	IRF540NPBF	15.61	(10.23+5.38)，含焊接在洞洞板上
续流二极管	ES2AA-13-F	4.21	肖特基，提升效率
电解电容	470uF 电解电容	1.87	输入输出滤波
自举/滤波电容	1nF / 1uF 电解电容	0.00	从实验室/别处找到
薄膜电容	470nF	4.82	前期误买（未用上）
电阻若干	各类色环电阻	10.53	6.57+3.96，含分压电阻与限流
插座与排针	2.54mm 排母/排针	2.58	电源与引线连接
洞洞板 (双面)	双面喷锡洞洞板	11.21	初期打样焊接验证
洞洞板 (单面)	单面洞洞板	3.73	备用/失败品
测试小灯泡	12V 小灯泡	3.72	简易负载测试
实际采购总支出		约 90.00 元	含运费与部分损耗件

5.2 损耗与损坏记录

在项目开发过程中，因购买失误、焊接失败及实验损耗，产生了以下额外支出：

- **购买失误：**初期误将 470nF 薄膜电容当作所需的自举电容购买，花费 4.82 元，未实际投入使用。
- **焊接损耗：**在洞洞板测试阶段，由于反复焊接与测试，导致 1 颗 IRF540N 功率 MOS 管及部分排针、电阻报废无法二次使用。
- **验证平台：**为了在 PCB 打样前验证电路原理，先后购买了双面喷锡洞洞板（11.21 元）和单面洞洞板（3.73 元），部分成功部分用于失败测试。
- **备用驱动芯片：**直插封装 IR2103 驱动芯片（3.12 元）主要用于洞洞板排针焊接，为损耗备用。

6 个人贡献

6.1 报告撰写

负责了本课程设计报告的整体排版与撰写工作，完成了项目背景调研、工作原理分析、硬件设计说明、软件算法讲解、测试数据统计以及最终的项目总结与反思。

6.2 编码工作

独立完成了 STM32CubeMX 的配置工作（时钟树、定时器 PWM、ADC 触发源及 DMA 配置）。在 STM32CubeIDE 中编写了主程序框架，并独立实现了基于硬件触发的 ADC 采样、DMA 数据搬运以及闭环 PID 控制算法代码的编写与调试。解决了初期在 100kHz 中断中进行 ADC 轮询导致 CPU 死机的问题。

6.3 硬件连接与调试

1. 完成了洞洞板的手工布局、焊接与初步测试。2. 使用嘉立创 EDA 辅助负责硬件设计的队友完成了 PCB 原理图绘制与实物排线布局。3. 焊接了最终的 PCB 成品板。4. 辅助参与了整套硬件系统的万用表测量、示波器观测以及电源负载测试。

6.4 资本投入

和队友共同承担了本次项目的硬件采购费用与 PCB 打样费用，实际总计支出约 30 元人民币。

6.5 开发问题描述与解决方案

- **问题描述 1:** 空载时输出正常，一加上负载 (44Ω)，电压迅速从 12V 掉压至 10.5V。
- **解决方案 1:** 经排查，原代码设置了 90% 的最大占空比限幅，导致 PID 在带载后无法继续增加占空比来补偿内阻压降。将限幅调整为 95%，并降低了 PID 的 Ki 系数，带载电压成功恢复并稳定在 11.5V 12V。
- **问题描述 2:** 焊接 PCB 板后，负载时有输出电压无输出电流。
- **解决方案 2:** 排查发现是因为自举电容误用了薄膜电容。更换为 1uF 电解电容，成功出现正常现象。

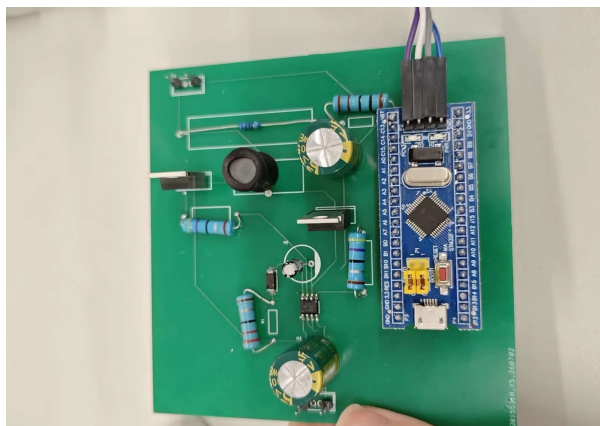


图 3 最终打样并焊接成功的 PCB 实物图

7 项目成果与反思

7.1 成果展示

本次项目成功设计并实现了一款基于 STM32 控制的高效率 15V 转 12V Buck 降压电路。主要成果包括：

- 成功完成 PCB 的绘制、打样与焊接。
- 实现了基于 DMA + 硬件触发的高实时性电压采样。
- 实现了稳定的数字 PID 闭环控制，能够在负载波动时快速调整。
- 峰值转换效率高达 92%，远超课设要求的 90% 优秀标准。

7.2 项目评估

成功之处：

- **高效率达成：**通过电感选型优化（100uH）和软件放开占空比上限，实测效率 92%，证明软硬件配合良好。
- **代码架构可靠：**硬件触发 + DMA 配合主循环 PID 的做法，避免了高频中断资源耗尽的问题，系统运行极其稳定。

不足与局限：

- 为了赶进度和追求极致的效率，未能完成扩展要求中的 OLED 显示、旋钮调压以及蓝牙无线遥控功能。

- 受限于 15V 输入电压下大电流时的线路内阻，44Ω 负载下电压轻微掉到 11.5V，未完美锁定在 12.000V。

7.3 个人反思与改进方向

通过本项目，我深入掌握了 Buck 电路的工作原理和开关电源设计要点。在遇到“PCB 板无法带载”、“占空比受限导致掉压”等问题时，培养了从硬件原理和代码逻辑双线排查故障的能力。特别是学会了 STM32 中 DMA 与中断的配合使用方法，这对后续开发其它嵌入式电源非常有帮助。

未来改进方向：

- **硬件升级：**将非同步整流升级为 **同步整流**（使用双 MOS 管替代续流二极管），理论上效率可突破 95% 甚至更高。
- **功能扩展：**增加 0.96 寸 OLED 屏幕，显示当前电压、电流及设定值；加入蓝牙模块，实现手机 App 远程监控调压。

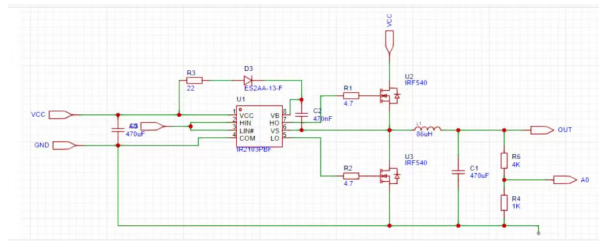


图 4 电路原理图

A 附录

A.1 设计图纸

完整电路原理图及 PCB 走线图如上下所示。

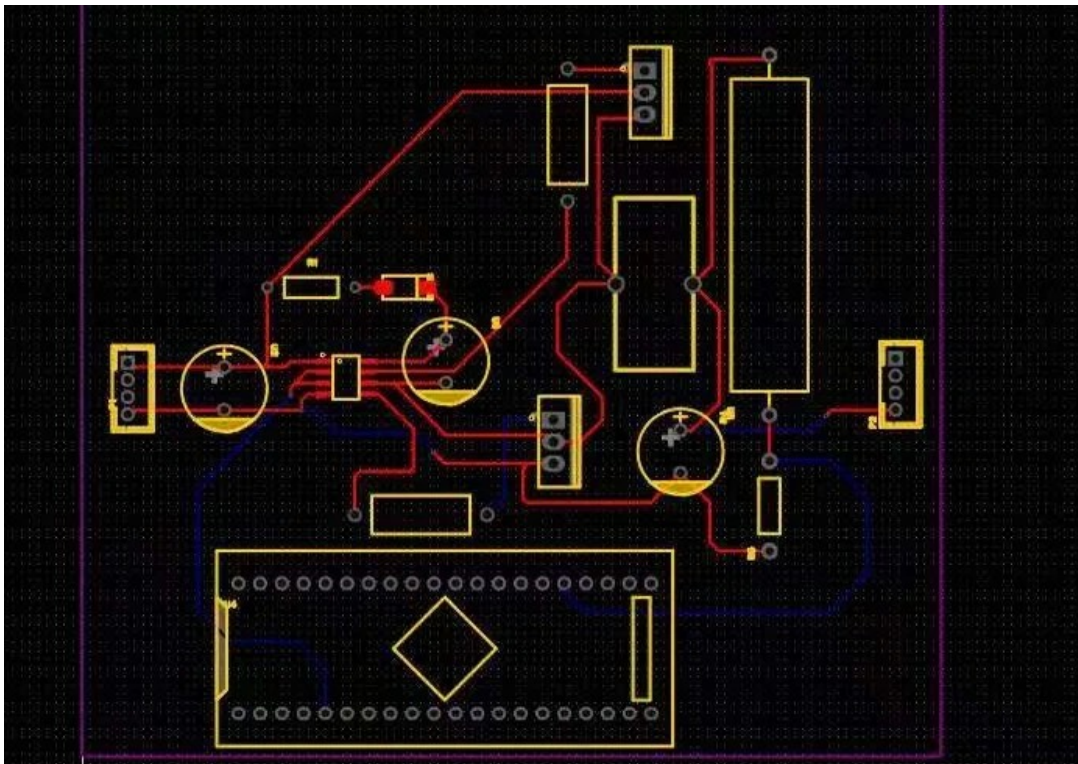


图 5 PCB 布局布线设计图

A.2 完整代码清单

本项目核心的底层驱动、主循环及 PID 控制算法代码如下（基于 STM32 HAL 库实现）：

```
/* USER CODE BEGIN Header */
```



```
/**
*****
* @file          : main.c
* @brief         : Main program body
* @attention     : 15V转12V Buck电路（硬件触发+DMA，100kHz零中断负载）
*****
*/
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"
#include "adc.h"
#include "dma.h"
#include "i2c.h"
#include "tim.h"
#include "gpio.h"

/* Private variables -----*/
/* USER CODE BEGIN PV */
// ===== ADC DMA 采集变量 =====
#define ADC_BUFFER_SIZE 1
uint16_t adc_buffer[ADC_BUFFER_SIZE]; // DMA 硬件自动写入的 ADC 值
volatile uint8_t adc_complete_flag = 0; // DMA 传输完成标志（数据更新标志）
uint16_t current_adc_value = 0; // 主循环中实际参与计算的 ADC 值

// ===== PID控制变量 =====
float Set_Voltage = 12.0f; // 目标电压：12V
float Kp = 0.3f; // 降低比例系数，防超调震荡
float Ki = 0.05f;
float Kd = 0.01f;
float Integral = 0.0f; // 积分累积值
float Last_Error = 0.0f;
uint32_t Duty_CCR = 540; // 初始占空比：540/720 = 75%
```

```
/* USER CODE END PV */

void PID_Control_Update(float feedback_voltage);

// ===== DMA 传输完成回调 =====
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if(hadc->Instance == ADC1)
    {
        adc_complete_flag = 1; // 仅置位标志，告诉主循环更新了数据
    }
}

// ===== 定时器中断回调（全部留空！） =====
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    // 【重要留空】：不要在这里写 HAL_ADC_Start 或 PID!
}

// ===== 主函数 =====
int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_GPIO_Init();
    MX_DMA_Init();
    MX_ADC1_Init();
    MX_TIM1_Init();
    MX_I2C1_Init();
    MX_TIM2_Init();
    MX_TIM3_Init();
```

```
/* USER CODE BEGIN 2 */
HAL_ADCEx_Calibration_Start(&hadc1);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 540);
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, ADC_BUFFER_SIZE);
/* USER CODE END 2 */

while (1)
{
    if (adc_complete_flag == 1)
    {
        adc_complete_flag = 0;
        current_adc_value = adc_buffer[0];

        // 分压比换算 (4k+1k 应为 *5.0)
        float feedback_voltage = (float)current_adc_value * 3.3f / 4096.0f * 5.0f;

        PID_Control_Update(feedback_voltage);
    }
}

// =====
// PID控制函数
// =====
void PID_Control_Update(float feedback_voltage)
{
    float Error;
    float P_term, I_term, D_term;
    float Output;
    float duty_float;
```

```
uint32_t ccr_value;

Error = Set_Voltage - feedback_voltage;

P_term = Kp * Error;
Integral += Error;
if(Integral > 50.0f) Integral = 50.0f;
if(Integral < -50.0f) Integral = -50.0f;
I_term = Ki * Integral;

D_term = Kd * (Error - Last_Error);
Last_Error = Error;

Output = P_term + I_term + D_term;

duty_float = 0.75f + Output / 100.0f;

if(duty_float > 0.90f) duty_float = 0.90f;
if(duty_float < 0.10f) duty_float = 0.10f;

ccr_value = (uint32_t)(duty_float * 720.0f);
if(ccr_value > 648) ccr_value = 648;
if(ccr_value < 72) ccr_value = 72;
Duty_CCR = ccr_value;

__HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, Duty_CCR);
}

void Error_Handler(void)
{
    __disable_irq();
    while (1)
```

```
{  
}  
}
```

A.3 参考文献

参考文献

- [1] 李志忠, 周贤中. 嵌入式系统设计及其项目开发. 西安电子科技大学出版社, 2026.
- [2] STMicroelectronics. *STM32F4xx Reference Manual* (RM0090), Rev 18, 2023.
- [3] Sanjaya Maniktala, *Switching Power Supplies A to Z*. Newnes, 2012.